

NASA Cosmic Log ETL Pipeline

Multi-Engine NoSQL Benchmarking and Reporting

Course: NoSQL Database Systems

Name	Roll Number
Rutul Patel	IMT2022021
Tanish Pathania	IMT2022049
Mohit Naik	IMT202076
Ananthakrishna K	IMT202206

May 17, 2026

Contents

1	Introduction	2
2	Problem Statement	2
3	System Architecture	2
3.1	Unified Logging & Observability	2
4	Implementation & Contributions	2
4.1	MongoDB Pipeline - Tanish Pathania	3
4.2	Apache Pig Pipeline - Mohit Naik	3
4.3	Apache Hive Pipeline - Rutul Patel	3
4.4	MapReduce Pipeline - Ananthakrishna K	3
5	Analytics and Reporting	3
6	Performance Metrics & Benchmarking	4
6.1	Cross-Pipeline Performance Leaderboard	4
6.2	Granular Query Telemetry	4
7	Conclusion	5

1 Introduction

Processing massive web server logs is a cornerstone of modern data engineering. The NASA Kennedy Space Center logs provide a rich dataset of HTTP requests from 1995, representing the kind of semi-structured data that NoSQL databases were designed to handle. This project implements a unified ETL (Extract, Transform, Load) framework to parse these logs and analyze them using four different distributed computing and storage technologies.

2 Problem Statement

Relational databases often encounter bottlenecks when horizontal scaling is required for massive ingestion of text-based log files. The primary objective of this project is to build a pluggable architecture that can:

- Ingest and parse complex HTTP log strings.
- Perform distributed aggregations for traffic analysis, resource identification, and error rate tracking.
- Compare the performance and developer experience across MongoDB, Apache Pig, Apache Hive, and MapReduce.

3 System Architecture

The system follows a modular design pattern. A central Java application serves as the orchestrator, utilizing a `LogParser` to transform raw text into `NasaLogRecord` objects. These objects are then fed into specific `Pipeline` implementations.

3.1 Unified Logging & Observability

To ensure a fair comparison, we implemented a centralized metadata store in **PostgreSQL**. Every execution run is assigned a unique `run_id`, and the system tracks:

- Ingestion time per batch.
- Malformed record counts.
- Query execution time for specific analytics.

4 Implementation & Contributions

Each team member was responsible for implementing one primary pipeline, ensuring end-to-end functionality from data loading to query execution.

4.1 MongoDB Pipeline - Tanish Pathania

Rutul implemented the document-oriented path using **MongoDB 6**. This involved:

- Mapping log records to BSON documents.
- Utilizing the MongoDB Aggregation Framework for high-performance grouping.
- Implementing complex queries for hourly error rates using conditional operators within the aggregation pipeline.

4.2 Apache Pig Pipeline - Mohit Naik

Tanish focused on the scripting abstraction layer for Hadoop using **Apache Pig**. Key tasks included:

- Writing Pig Latin scripts to load TSV data.
- Implementing deduplication logic for distinct host counts.
- Integrating the **PigServer** library within the Java application for seamless execution.

4.3 Apache Hive Pipeline - Rutul Patel

Mohit developed the SQL-on-Hadoop path using **Apache Hive**. This contribution involved:

- Designing the Hive schema for raw log storage.
- Implementing HQL (Hive Query Language) scripts for analytical queries.
- Managing JDBC connections to the HiveServer2 container.

4.4 MapReduce Pipeline - Ananthakrishna K

Ananthakrishna implemented the low-level distributed processing path using **Native Hadoop MapReduce**. This complex task involved:

- Creating custom **Writable** classes for keys and values.
- Implementing specific Mappers and Reducers for each of the three analytical queries.
- Managing the HDFS-local file system interactions within the containerized environment.

5 Analytics and Reporting

Regardless of the engine used, the system generates three key reports:

1. **Daily Traffic Analysis:** Counts total requests and bandwidth per status code per day.
2. **Top Resource Utilization:** Identifies the most frequently accessed paths and bandwidth hogs.
3. **Error Rate Monitoring:** Tracks the percentage of 4xx and 5xx errors per hour.

6 Performance Metrics & Benchmarking

The unified observability layer built on PostgreSQL successfully captured the execution telemetry for each pipeline. The benchmark was run on the complete NASA dataset (approx. 3.46 million records) with a batch size of 1,000.

6.1 Cross-Pipeline Performance Leaderboard

The total runtime reflects the entire end-to-end execution, including data ingestion, transformation, and query execution for all three analytics suites.

Rank	Pipeline	Total Runtime (ms)
1	Apache Hive	65,591
2	MongoDB	70,569
3	Apache Pig	124,230
4	MapReduce	248,993

Table 1: End-to-End Pipeline Performance Comparison

6.2 Granular Query Telemetry

Breaking down the performance by specific analytical query reveals the relative strengths of each engine’s aggregation framework.

Pipeline	Q1: Daily Traffic (ms)	Q2: Top Resources (ms)	Q3: Hourly Errors (ms)
MongoDB	3,997	26,229	4,618
Apache Hive	11,929	27,429	9,775
Apache Pig	29,800	52,962	31,181
MapReduce	76,791	87,492	75,399

Table 2: Query Execution Times by Analytical Task

Key Observations:

- **Hive’s Overall Lead:** While MongoDB executed specific queries faster, Hive had the lowest overall total runtime (65.5s), indicating highly efficient batch insertion and data loading mechanics compared to MongoDB’s document insertion overhead.
- **MongoDB’s Query Dominance:** MongoDB’s Aggregation Pipeline proved to be exceptionally fast for targeted queries, executing the Daily Traffic query in just 3.9 seconds, compared to Hive’s 11.9 seconds.
- **MapReduce Overhead:** Native MapReduce exhibited the longest execution times across all metrics, highlighting the massive developer and performance overhead introduced by managing low-level HDFS I/O and job scheduling compared to higher-level abstractions like Hive.

7 Conclusion

The project successfully demonstrates the versatility of NoSQL systems in handling cosmic-scale log data. While MongoDB offered the most intuitive developer experience, MapReduce provided the deepest control over the distributed execution flow. The unified observability layer proved essential in benchmarking these systems accurately.